

# Generic external axis over Modbus TCP — protocol

One register map for any external axis a PLC fronts: a linear track carrying the robot (Pamet: Kinco PLC → Lichuan servo), a rotary positioner carrying the part, or any drive behind a PLC that can serve Modbus TCP. rw2 owns the axis through this map alone. The robot controller's own external-axis coupling is never used (owner ruling 2026-09-28/29). The map is the contract between the driver in this repository and the PLC program; both sides are written against this file.

Lineage: epic-065 external-axis-motion (Delta VFD-C2000 positioner, `delta-positioner` `DeltaRegisters.cs`, FR-11 dead-commander watchdog) and epic-065 feature-007 (Pamet track directly from the Kinco PLC).

## Transport

| Item                         | Value                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Transport                    | Modbus TCP, port 502 (configurable)                                                                                                                                                                                                                                                                                                                     |
| Unit id                      | 1 (configurable)                                                                                                                                                                                                                                                                                                                                        |
| Register type                | Holding registers only (FC03 read, FC06/FC16 write). No coils: one register file is simpler to map in every PLC.                                                                                                                                                                                                                                        |
| Block placement              | Two blocks. <b>Command block</b> at base <code>C</code> (default <code>0</code> ), <b>status block</b> at base <code>S</code> (default <code>100</code> ). Offsets inside a block are fixed; only the two bases are configurable, on both sides, for a PLC whose register file cannot start at 0/100. The addresses below are written for the defaults. |
| Word order for 32-bit values | <b>low word first</b> (register <code>n</code> = bits 0–15, register <code>n+1</code> = bits 16–31), two's complement; each 16-bit register big-endian on the wire (Modbus standard)                                                                                                                                                                    |
| Position unit                | 0.001 mm for a linear axis, 0.001° for a rotary axis (the driver's configured kind decides; the map is the same)                                                                                                                                                                                                                                        |
| Velocity unit                | 0.001 unit/s; acceleration 0.001 unit/s <sup>2</sup>                                                                                                                                                                                                                                                                                                    |
| Consistency                  | The PLC answers one FC03 read of the whole status block (15 registers) from <b>one scan's image</b> , and applies one FC16 write before or after a scan, never in the middle of one.                                                                                                                                                                    |
| Map version                  | register 114 = <b>1</b> for this document                                                                                                                                                                                                                                                                                                               |

## Command block — written by the driver, read by the PLC (holding registers `C+0 ... C+11`)

| Addr | Name    | Type     | Meaning                                                                                                                                                                                                                                                                                                                                                                    |
|------|---------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0    | Command | bitfield | bit 0 <b>Enable</b> (level: 1 = servo on). bit 1 <b>Home</b> , bit 2 <b>MoveAbsolute</b> , bit 3 <b>MoveVelocity</b> , bit 4 <b>Stop</b> , bit 5 <b>Reset</b> — edge-triggered: the PLC acts on 0→1 and the driver clears the bit after <code>CommandAck</code> echoes <code>CommandSeq</code> . Stop has priority over every other bit in the same word. Bits 6–15 are 0. |

| Addr | Name           | Type   | Meaning                                                                                                                                                                                                                  |
|------|----------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | CommandSeq     | uint16 | Incremented by the driver with every <b>command write</b> — a write that sets an edge bit or changes Enable (never 0; 65535 wraps to 1). The write that only clears edge bits after an ack does <b>not</b> increment it. |
| 2–3  | TargetPosition | int32  | Target for MoveAbsolute, in position units.                                                                                                                                                                              |
| 4–5  | Velocity       | int32  | Speed for MoveAbsolute (magnitude) and MoveVelocity (signed: sign = direction).                                                                                                                                          |
| 6–7  | Acceleration   | int32  | Ramp for the next move; 0 = PLC default.                                                                                                                                                                                 |
| 8    | Heartbeat      | uint16 | FR-11: the driver writes an incrementing value (1...65535, never 0) at $\geq 5$ Hz for the whole connection lifetime. The driver's default is 10 Hz.                                                                     |
| 9    | LeaseOwner     | uint16 | FR-11 advisory lease: 0 = unowned; else the station-unique owner id of the driver holding the axis.                                                                                                                      |
| 10   | WatchdogFault  | uint16 | 0 healthy; 1 = the PLC tripped the watchdog. <b>Cleared only by the driver writing 0.</b> Written by the PLC on trip.                                                                                                    |
| 11   | WatchdogTrips  | uint16 | Trip counter since PLC power-up. PLC-owned, read-only for the driver.                                                                                                                                                    |

## Status block — written by the PLC, read by the driver (holding registers S+0 ... S+14)

| Addr    | Name           | Type     | Meaning                                                                                                                                                                                                                          |
|---------|----------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 100     | State          | uint16   | 0 Disabled · 1 Standstill · 2 Homing · 3 DiscreteMotion · 4 ContinuousMotion · 5 (reserved) · 6 Stopping · 7 ErrorStop — the numbers of the SDK <code>AxisState</code> enum. Any other value is read by the driver as ErrorStop. |
| 101     | Flags          | bitfield | bit 0 Homed · bit 1 InPosition · bit 2 LimitMin · bit 3 LimitMax · bit 4 HomeSensor · bit 5 DriveReady · bit 6 Moving                                                                                                            |
| 102–103 | ActualPosition | int32    | Position units. Valid only while <code>Homed</code> is set (an absolute encoder keeps it across power cycles; the map does not care which).                                                                                      |
| 104–105 | ActualVelocity | int32    | Signed, 0.001 unit/s.                                                                                                                                                                                                            |
| 106     | FaultCode      | uint16   | 0 none · 1 drive fault · 2 limit switch tripped · 3 following error · 4 watchdog · 5 homing failed · 6 communication to drive lost · 7 safety stop (E-stop chain, guard) · 100+ vendor-specific (documented per PLC).            |
| 107     | CommandAck     | uint16   | Echo of the last accepted <code>CommandSeq</code> .                                                                                                                                                                              |
| 108–109 | TravelMin      | int32    | Machine limit, published by the PLC. The driver refuses targets outside <code>TravelMin..TravelMax</code> ; it never clamps.                                                                                                     |
| 110–111 | TravelMax      | int32    |                                                                                                                                                                                                                                  |
| 112–113 | MaxVelocity    | int32    | Machine limit, published by the PLC. The driver refuses faster speeds.                                                                                                                                                           |
| 114     | MapVersion     | uint16   | 1. The driver reads the status block before its first write and refuses to attach to any other value, writing nothing.                                                                                                           |

Limits come from the machine through this block. If a PLC cannot publish them ( `TravelMin` , `TravelMax` and `MaxVelocity` all zero), the driver uses the values from its own configuration and logs that it did; it never invents a number. When they are published they must satisfy `TravelMin < TravelMax` and `MaxVelocity > 0`; any other combination (a partial publication included) makes the driver refuse to attach.

## Command semantics

---

- **Handshake.** Before the command write the driver writes the parameters it needs ( `TargetPosition` , `Velocity` , `Acceleration` , registers 2–7) in one FC16, then `Command` and `CommandSeq` (registers 0–1) in a second FC16 — so the PLC never sees an edge before its parameters. The PLC accepts a command word whose `CommandSeq` differs from `CommandAck` and copies `CommandSeq` into `CommandAck` **in the same scan in which it enters the resulting state** (Homing, DiscreteMotion, ContinuousMotion, Stopping, Disabled, Standstill). A status read that shows the new ack therefore also shows the command's state, never the state from before it.
- **Acknowledge:** a command is accepted when `CommandAck == CommandSeq` . The driver waits  $\leq 500$  ms for the ack; no ack  $\rightarrow$  SDK `CommunicationLost` for the command (the heartbeat keeps running); the driver clears the edge bit it set.
- **Sequence at attach:** the driver reads `CommandAck` and continues from `CommandAck + 1` , so a new driver never issues a sequence number the PLC already acknowledged.
- **Enable** (level): `Command.bit0 = 1`  $\rightarrow$  PLC energises the drive; State leaves Disabled for Standstill. `bit0 = 0`  $\rightarrow$  drive off, State = Disabled. After a watchdog trip or any entry into ErrorStop the PLC ignores bit 0 until a Reset, and after the Reset it energises only on a fresh 0 $\rightarrow$ 1 of bit 0 — an axis never re-energises by itself. A move with Enable low is ignored and answered by `FaultCode = 0` , State = Disabled ; the driver rejects it before writing (SDK `MotionError.Busy` ).
- **Home** (edge): PLC runs its own homing sequence; State = Homing until done, then Standstill with `Homed` . The driver treats Home as callable from Disabled (it is the energise act: the driver sets Enable first). Failure  $\rightarrow$  ErrorStop, `FaultCode = 5` .
- **MoveAbsolute** (edge): requires `Homed` ; PLC moves to `TargetPosition` at `Velocity` ; State = DiscreteMotion, then Standstill with `InPosition` . Target outside travel  $\rightarrow$  the driver refuses (SDK `OutOfRange` ) and writes nothing.
- **MoveVelocity** (edge): continuous motion at signed `Velocity` until Stop or a limit; State = ContinuousMotion. While `Homed` , reaching `TravelMin` / `TravelMax` is a controlled stop to Standstill (not a fault). A limit switch is always a fault: ErrorStop, `FaultCode = 2` .
- **Stop** (edge, priority): accepted in every state, Homing included; decelerate and hold; State = Stopping  $\rightarrow$  Standstill. From Disabled or Standstill it is acknowledged and changes nothing. The driver sends Stop on the priority lane (see FR-11) and expects motion to cease within 200 ms of the write.
- **Reset** (edge): clears `FaultCode` and leaves ErrorStop  $\rightarrow$  Disabled (Enable must be re-asserted, see Enable). Does not clear `WatchdogFault` (register 10 is cleared explicitly) and does not touch `Homed` . From any state other than ErrorStop it is acknowledged and changes nothing.
- **Limit switch recovery:** after a Reset with a limit switch still active, the PLC accepts motion only in the direction away from that switch.

## FR-11 — dead-commander watchdog and advisory lease (verbatim from epic-065)

---

Motion must cease when the commander **dies**, not only when it calls Stop. Modbus TCP has no liveness semantics; a dead commander is indistinguishable from an idle one. Therefore:

- The driver runs a **connection-lifetime heartbeat** per axis, started at connect, stopped deliberately at disconnect, independent of motion, writing `Heartbeat` at  $\geq 5$  Hz. Heartbeat traffic shares the channel with move traffic; heartbeat deferral is **bounded at 200 ms** so a long move cannot starve the beat.
- The PLC **arms** the watchdog on the **first change** of `Heartbeat` it observes while `WatchdogFault = 0` and `LeaseOwner  $\neq$  0` (so an idle PLC is not faulted one second after boot), then **trips** when `Heartbeat` has not changed for **1 s**: Enable dropped, velocity zeroed, `State = ErrorStop`, `FaultCode = 4`, `WatchdogFault = 1`, `WatchdogTrips += 1`. A trip does **not** touch `Homed` (recovery = Reset + re-command, no re-home) and re-asserts the limit functions. After a trip the network disarms until `WatchdogFault` has been written 0 and a new beat arrives.
- **Clean release disarms.** When `LeaseOwner` is written 0 the watchdog disarms without tripping. A clean disconnect is: Stop  $\rightarrow$  Enable 0  $\rightarrow$  stop beating  $\rightarrow$  `LeaseOwner = 0`. `WatchdogTrips` therefore counts dead commanders only. A killed process cannot write 0, so the kill case is untouched.
- **Advisory lease:** before attaching, the driver reads `LeaseOwner`. 0  $\rightarrow$  take it (write own id). Own id  $\rightarrow$  reattach. Foreign id  $\rightarrow$  watch `Heartbeat` at the heartbeat interval; unchanged for one expiry window (1 s)  $\rightarrow$  the lease has expired, take it; still changing  $\rightarrow$  refuse (SDK `LeaseHeld`), keep watching until the configured lease timeout. Watching continuously rather than in discrete 1 s samples lets a successor attach  $\leq 1.1$  s after the incumbent's last beat. "Advisory" is stated plainly: Modbus has no compare-and-swap, so two drivers starting in the same window can both pass; the watchdog bounds the consequence. On a clean disconnect the driver writes `LeaseOwner = 0` if it still holds it.
- **At attach** the driver writes `WatchdogFault = 0` after winning the lease: a latched trip then belongs to a dead predecessor, and leaving it latched would leave the network disarmed for the new commander. `FaultCode = 4` and `ErrorStop` stay until a Reset, so the trip remains visible and distinguishable from a drive fault.
- Owner id is a station-unique non-zero 16-bit value from host configuration.
- Each heartbeat tick is: write `Heartbeat`, read registers 10–11, read the status block. That tick is the status cadence of the axis, moving or idle.
- Stop and cancellation take a **priority lane** on the channel that preempts queued move traffic ( $\leq 200$  ms to a halted axis).

## What the PLC program must do (checklist for the ladder / structured-text author)

---

1. Serve holding registers C+0...C+11 and S+0...S+14 on the configured unit id and port; refuse nothing, ignore writes to PLC-owned registers (11, 100–114).
2. Publish `MapVersion = 1`, `TravelMin/Max`, `MaxVelocity` from the axis parameters at start-up.
3. Mirror the drive: `State`, `Flags`, `ActualPosition`, `ActualVelocity`, `FaultCode` every PLC cycle; serve a status-block read from one scan's image.
4. Execute the command semantics above; echo `CommandSeq` into `CommandAck` in the scan that enters the command's state.
5. Implement the watchdog exactly as FR-11 states (arm on first change with a lease held, 1 s stall, trip actions, disarm until cleared, disarm on `LeaseOwner = 0`).
6. Pass the conformance checks below, CHK-01...CHK-16 (motion checks at commissioning, with an operator present).
7. Keep the project in a git repository next to this file's copy (epic-065 risk R-1: an unversioned PLC program is a drive that answers every register and moves nothing after replacement).

## Scope of map version 1

---

- One axis per block pair. A PLC serving two axes places a second block pair at other bases and is configured as two devices.
- A rotary axis has limited travel: `ActualPosition` is absolute and never wraps. Endless rotation is not in version 1.

## Conformance checks

This checklist is executable. Two tools run it verbatim against a PLC, with no `rw2` involved:

- the C# test app, `dotnet ModelingEvolution.GenericAxis.TestApp.dll --check <host>[:port] ...`;
- the Python tool, `python -m generic_axis_check <host>[:port] ...` (in `python/generic_axis_check/`).

The simulator must pass the full list, motion included, in CI. The table below is the single source of truth. A check id missing from either tool, or present in a tool but not here, is a defect, and each tool has a test that compares its ids with this table.

### Command line (identical in both tools)

| Argument                                                        | Default  | Meaning                                                                                                                                                                                                                                                   |
|-----------------------------------------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>&lt;host&gt;[:port]</code>                                | port 502 | The PLC.                                                                                                                                                                                                                                                  |
| <code>--unit N</code>                                           | 1        | Unit id.                                                                                                                                                                                                                                                  |
| <code>--command-base N</code> ,<br><code>--status-base N</code> | 0, 100   | Block bases.                                                                                                                                                                                                                                              |
| <code>--owner-id N</code>                                       | 65535    | The checker's lease id. CHK-11 uses 65534 as the foreign id. Ids 65534–65535 are reserved for conformance tools; stations never use them.                                                                                                                 |
| <code>--allow-motion</code>                                     | off      | Runs CHK-12...CHK-16. Without it they are SKIPPED. <b>Only with an operator at the machine and the travel clear.</b>                                                                                                                                      |
| <code>--tolerance X</code>                                      | 0.1      | Position check threshold in axis units (CHK-13). This is a checker threshold, not a machine number.                                                                                                                                                       |
| <code>--report PATH</code>                                      | none     | <code>*.md</code> : writes the Markdown report there and the JSON report next to it as <code>*.json</code> . <code>*.json</code> : writes the JSON report only. Any other extension is a usage error (exit 2). The Markdown report always goes to stdout. |

Exit codes: 0 = no FAIL (SKIPPED allowed) · 1 = at least one FAIL · 2 = usage error · 3 = refused to start (the axis is held by a live foreign lease, meaning another commander such as `rw2` is attached; stop it first).

### Rules for every run

- **Pre-flight.** Read `LeaseOwner` and watch `Heartbeat` for 1 s. If a foreign owner is beating, exit 3, write nothing, and report every check SKIPPED.
- **Order.** Checks run in id order. A check whose prerequisite FAILED or was SKIPPED is SKIPPED, and its message names the prerequisite.
- **Timing.** Timing checks poll the status block every **20 ms**. Every duration is measured from the completion of the triggering write to the first read that shows the effect, and is reported in ms.
- **Units.** All positions and velocities are raw register values ÷ 1000, in the PLC's axis unit (mm or °).

- **Cleanup, always, even after a FAIL or Ctrl-C:** Stop edge if State is 2, 3 or 4 · clear edge bits · Enable 0 · `WatchdogFault = 0` if the checker caused a trip · `LeaseOwner = 0` if it holds the checker's id. Each cleanup write is logged in the report.
- **Lease and beat between checks.** From CHK-06 onwards the checker holds the lease under its own id and beats, except where a check says it stops.
- **Each check restores.** Every check ends with the axis in State 0 or 1, no latched fault, the lease held and the beat running: Reset, `WatchdogFault = 0` and re-take as needed. If it cannot restore, it FAILs with the reason, and every later check is SKIPPED.
- **Beat.** Whenever a check says "beat", the checker writes `Heartbeat` every 100 ms (1...65535, never 0) from its own loop, not through a driver.

## The checks

| Id     | Title                                                | Protocol section                             | Needs | Procedure                                                                                                                                                                | PASS when                                                                                                                                                                                                                             |
|--------|------------------------------------------------------|----------------------------------------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CHK-01 | Transport and unit                                   | Transport                                    | —     | TCP connect (a budget of $\leq 3$ s including the tool's own retry; a budget, not a timing threshold), then FC03 of S+0...S+14 on the unit.                              | Connected, and the read answers with no Modbus exception.                                                                                                                                                                             |
| CHK-02 | Map version                                          | Status block                                 | 01    | Read S+14.                                                                                                                                                               | <code>MapVersion == 1</code> .                                                                                                                                                                                                        |
| CHK-03 | Machine limits published                             | Status block, "Limits come from the machine" | 02    | Read S+8...S+13.                                                                                                                                                         | Not all zero, <code>TravelMin &lt; TravelMax</code> , <code>MaxVelocity &gt; 0</code> . The values are reported.                                                                                                                      |
| CHK-04 | Status mirror cadence                                | Status block; FR-11 tick                     | 02    | 30 status-block reads, one every 100 ms.                                                                                                                                 | All 30 answer, the slowest round trip is $\leq 100$ ms, and <code>State</code> $\in \{0,1,2,3,4,6,7\}$ in every read.                                                                                                                 |
| CHK-05 | 32-bit word order and driver ownership of parameters | Transport (word order); Command block        | 02    | Write C+2...C+3 = <code>[0x0002, 0x0001]</code> (65 538), read back. Write -2 as <code>[0xFFFE, 0xFFFF]</code> , read back. Wait 1 s and read again. No edge bit is set. | Both values read back exactly, and are unchanged after 1 s (the PLC does not write driver-owned registers). The PLC's <i>interpretation</i> of the order is proven by CHK-03 (sane limits) and CHK-13 (it arrives where it was sent). |
| CHK-06 | Enable handshake (level)                             | Command semantics: Handshake, Enable         | 02    | Precondition State 0 or 1. Take the lease (checker id), beat. Write <code>[Enable, seq+1]</code> , then after the ack <code>[0, seq+2]</code> .                          | Each ack arrives in $\leq 500$ ms. State is 1 within 5 s after Enable 1 and 0 within 5 s after Enable 0. Ack ms and state ms are reported. Energises the drive and commands no motion.                                                |

| Id     | Title                                              | Protocol section                      | Needs               | Procedure                                                                                                                                                                                                                                                                                 | PASS when                                                                                                                                                                                                                               |
|--------|----------------------------------------------------|---------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CHK-07 | Reset handshake (edge)                             | Command semantics: Reset, Acknowledge | 06                  | From State 0 write [Reset, seq+1]. After the ack, clear the edge [0, seq+1].                                                                                                                                                                                                              | Ack in $\leq 500$ ms, State stays 0 and FaultCode stays 0 (Reset outside ErrorStop is a no-op).                                                                                                                                         |
| CHK-08 | Watchdog trips on a stalled beat                   | FR-11                                 | 06                  | Hold the lease, WatchdogFault = 0, beat for 2 s, then stop beating. Keep polling.                                                                                                                                                                                                         | WatchdogFault == 1, WatchdogTrips +1, State == 7, FaultCode == 4, all within <b>1.0–1.5 s</b> of the last beat. The trip time is reported.                                                                                              |
| CHK-09 | Watchdog disarms after a trip and re-arms on clear | FR-11                                 | 08                  | Setup: first trip the watchdog as in CHK-08 (lease held, WatchdogFault = 0, beat 2 s, stop, wait $\leq 1.5$ s; no trip → FAIL "setup: no trip"). Then, without clearing, beat 1 s: no second trip is counted. Then Reset edge, WatchdogFault = 0, beat 2 s (must not trip), stop beating. | No trip while latched. No trip while beating. A second trip within 1.0–1.5 s with WatchdogTrips +1. Recovery afterwards is Reset plus WatchdogFault = 0.                                                                                |
| CHK-10 | Clean release disarms                              | FR-11 "Clean release disarms"         | 08                  | Beat 2 s, write LeaseOwner = 0, stop beating, wait 2 s.                                                                                                                                                                                                                                   | No trip: WatchdogFault == 0 and the trip count is unchanged.                                                                                                                                                                            |
| CHK-11 | Advisory lease                                     | FR-11 "Advisory lease"                | 02                  | (a) LeaseOwner = 0 → the checker's lease client takes it and reads back its id, then releases. (b) Write LeaseOwner = 65534 and beat as that incumbent. The checker's lease client, with a 3 s timeout, must refuse. (c) Stop the incumbent's beat while the client watches.              | (a) Read-back equals the checker id. (b) Refused with LeaseHeld naming 65534 after 3 s, and LeaseOwner is still 65534. (c) Taken within 2 s of the incumbent's last beat, with the time reported. Any trip caused by (c) is cleaned up. |
| CHK-12 | Home                                               | Command semantics: Home               | 06, -- allow-motion | Enable, then Home edge.                                                                                                                                                                                                                                                                   | Ack in $\leq 500$ ms. Then State == 1 with Homed within 120 s, and FaultCode == 0. Duration reported.                                                                                                                                   |
| CHK-13 | MoveAbsolute to TravelMin + 10                     | Command semantics: MoveAbsolute       | 03, 12              | Target TravelMin + 10, velocity 10 % of MaxVelocity, acceleration 0.                                                                                                                                                                                                                      | Ack with State == 3 in $\leq 500$ ms. Then State == 1 with InPosition, and $\text{abs}(\text{ActualPosition} - \text{target}) \leq \text{--tolerance}$ . The error and duration are reported.                                           |

| Id     | Title         | Protocol section                        | Needs | Procedure                                                                                                                                                                                       | PASS when                                                                                                                                                                                           |
|--------|---------------|-----------------------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CHK-14 | Stop mid-move | Command semantics: Stop; FR-11 priority | 13    | MoveAbsolute toward $\text{TravelMin} + (\text{TravelMax} - \text{TravelMin})/2$ at 10 %. Once $\text{abs}(\text{ActualVelocity}) \geq 90$ % of the commanded speed (or after 2 s), write Stop. | Ack in $\leq 500$ ms. $\text{ActualVelocity} == 0$ and $\text{State} == 1$ within <b>200 ms</b> of the Stop write. The time is reported.                                                            |
| CHK-15 | MoveVelocity  | Command semantics: MoveVelocity         | 13    | MoveVelocity at +1 % of $\text{MaxVelocity}$ (away from $\text{TravelMin}$ ) for 1 s, then Stop.                                                                                                | Ack with $\text{State} == 4$ , $\text{ActualVelocity} > 0$ during the run, then $\text{State} == 1$ after Stop within 200 ms.                                                                       |
| CHK-16 | Kill test     | FR-11                                   | 15    | MoveVelocity at +1 %, then stop beating. The connection stays open, and polling continues.                                                                                                      | Trip ( $\text{FaultCode } 4$ , $\text{State } 7$ ) within 1.0–1.5 s of the last beat. $\text{ActualVelocity} == 0$ within 200 ms of the trip. $\text{Homed}$ is still set. Both times are reported. |

## Report schema

JSON ( schema: "generic-axis-conformance/1" ). Both tools emit exactly these fields:

```
{
  "schema": "generic-axis-conformance/1",
  "mapVersion": 1,
  "tool": { "name": "generic-axis-check", "language": "python", "version": "1.0.0" },
  "target": { "host": "192.168.58.20", "port": 502, "unit": 1, "commandBase": 0,
"statusBase": 100 },
  "allowMotion": false,
  "startedAt": "2026-09-29T10:15:02Z",
  "finishedAt": "2026-09-29T10:15:31Z",
  "summary": { "result": "PASS", "pass": 11, "fail": 0, "skipped": 5 },
  "checks": [
    { "id": "CHK-08", "title": "Watchdog trips on a stalled beat", "section": "FR-11",
      "result": "PASS", "durationMs": 3140,
      "message": "trip after 1.12 s",
      "observed": { "tripAfterMs": 1120, "watchdogTrips": 3, "faultCode": 4,
"state": 7 } },
    { "id": "CHK-12", "title": "Home", "section": "Command semantics: Home",
      "result": "SKIPPED", "durationMs": 0, "message": "needs --allow-motion",
"observed": {} }
  ],
  "cleanup": [ "C+0 = 0x0000 (Enable 0)", "C+9 = 0 (release lease)" ]
}
```

- `result` is PASS, FAIL or SKIPPED. `summary.result` is FAIL if any check failed, otherwise PASS.

- `observed` keys are camelCase. Numbers are raw integers or durations in ms. `language` is `python` or `csharp`.

The Markdown report has three parts. A heading gives the tool, the target and the UTC time. A table follows with the columns `Id` | `Title` | `Result` | `Observed` | `Protocol` section. Last comes the cleanup list. The last line is `RESULT: PASS` or `RESULT: FAIL`.

## Reference: the Delta positioner ladder this generalises

---

```
M0 -> M1025 (RUN) + M1040 (servo on)
M4 -> edges: MOV 0/1 D1060 (speed/position mode); FREQ D110 D111 D112
M5 -> M1026 (direction: OFF = forward, ON = reverse)
M6 + edge X7 -> DSUB D1051 D120 D122; DMOV D1051 D120 (home latch)
D130 heartbeat · D131 lease owner · D132 latched watchdog fault · D133 trip counter
```

(`delta-positioner/src/RocketWelder.SDK.Devices.Motion.Delta/DeltaRegisters.cs`, `epic-065 current-state.md`.)